İZMİR UNIVERSITY
OF ECONOMICS

Sports Management Simulation Game

Final Report

Prepared by:

Şilan ATÇI	20230602008
Burak ÇALIŞKAN	20230602017
Beray KAYA	20230602045
Duru Eda CAN	20240602019

Instructor:
Doç. Dr. Kaya OĞUZ

Department of Computer Engineering

CE 216 Project

May 2026

Contents

Chapter 1

Introduction

1.1 Project Summary

The Sports Management Simulation Game is a desktop application where a single player takes the role of a sports team manager. The player chooses a sport (Football or Handball), creates a team, builds a squad, and competes through a full league season against AI-controlled opponents. The game simulates weekly match cycles including training, match preparation, a live match view with a half-time break, post-match effects (injuries and energy loss), and end-of-season standings.

The goal of this project was to apply object-oriented design principles, design patterns, and clean software architecture to a system complex enough to demonstrate the benefits of these practices. The project was completed in three milestones:

- **M1** – System design: UML class diagrams, use-case analysis, and architecture decisions.
- **M2** – Core implementation: domain layer, application layer, sport plugin layer, and unit tests.
- **M3 (this report)** – Complete system: JavaFX user interface, save/load system, second sport (Handball), coach system, and full integration.

1.2 Motivation

Sports management simulations combine several interacting systems such as scheduling, match simulation, player progression, standings management, and user interaction. Without a well-structured architecture, these responsibilities can quickly become tightly coupled and difficult to maintain or extend. This project demonstrates how layered architecture and object-oriented design principles can help manage that complexity effectively.

Chapter 2

System Overview

2.1 What the Game Does

Sports Manager is a turn-based sports management simulation game. The player takes the role of a manager and guides a team through a full league season against AI-controlled opponents. The game supports two sports (Football and Handball), each with its own league size, fixture length, and match simulation rules.

At the start of a new game, the player provides a manager name, team name, gender category (Male or Female), and sport type. The application then generates a complete league with AI teams, assigns players and a default coach to each team, and creates a round-robin fixture schedule for the season.

Each in-game week follows a fixed sequence:

1. **Dashboard**: Review the upcoming match, current league standings, coach status, and active injury reports.
2. **Training**: Choose a training intensity (Light, Medium, or Hard) that affects player condition, energy, and injury risk for the week.
3. **Squad Management**: Adjust the starting lineup and substitute bench before the match.
4. **Coach**: Optionally hire a higher-level coach to improve training multipliers and reduce match energy loss.
5. **Match**: Play through two halves. The simulation engine calculates each period's result using player condition, energy, tactics, and coach bonuses.
6. **Half-Time**: Change the team tactic before the second half begins.
7. **Post-Match**: View the final score, injuries sustained, energy changes, and reputation points earned.

When all match weeks are completed, the Season End screen displays the champion and

allows the player to begin the next season with the same squads and updated player states.

2.2 Supported Sports

- **Football**: 18 teams, double round-robin schedule (34 weeks), and 2 halves per match.
- **Handball**: 12 teams, double round-robin schedule (22 weeks), and 2 halves per match.

Chapter 3

System Architecture

3.1 Layered Architecture

The system is divided into four layers. Each layer has a clearly defined responsibility and communicates only with the layer directly below it. The Domain and Application layers were designed and implemented during the previous milestones (M1 and M2). This section focuses on the two major components completed in M3: the JavaFX UI layer and the Handball sport plugin. Together, these additions demonstrate that the architecture supports extension and integration as originally planned.

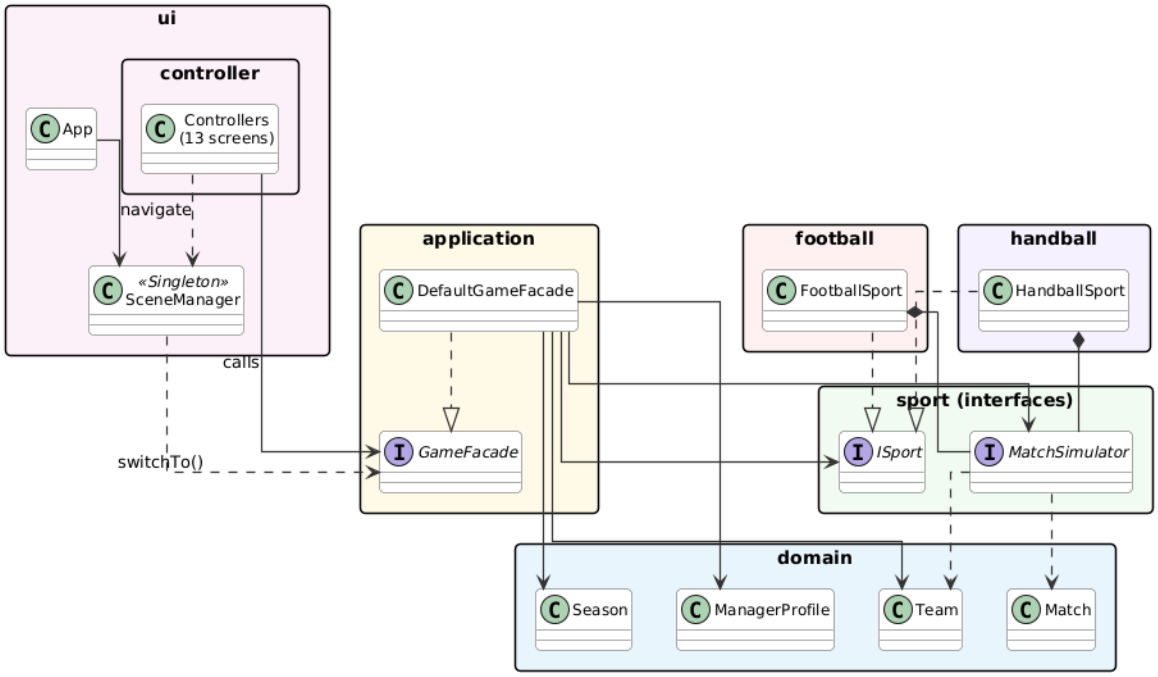

Figure 3.1: Four-layer architecture of the Sports Management Simulation Game

3.1.1 UI Layer

Package: `ui`, `ui.controller`

The UI layer is the largest new addition introduced in M3. It is built entirely with JavaFX and sits on top of the existing application layer without modifying its internal logic. The layer consists of three main components:

- `App`: The JavaFX entry point. It creates the primary Stage, passes it to SceneManager, and opens the main menu when the application starts.
- `SceneManager`: A Singleton responsible for managing screen transitions and holding the Stage reference. Controllers call switchTo("screen-name", facade), and the manager replaces the root of the existing Scene instead of creating a new Stage. Full details are provided in Section 4.3.1.
- `SoundManager`: A Singleton that loads ".m4a" audio files from the resources folder and plays audio feedback for events such as button clicks, victories, draws, game-over states, and championship celebrations.

All screens follow the same structural pattern: each screen has a dedicated controller in the ui.controller package that receives a GameFacade reference during construction. Controllers may read domain data objects (such as Player, Match, and StandingEntry) for display purposes, but all business operations are delegated through the GameFacade interface. Controllers never reference manager or generator classes directly, which helps keep business logic separated from the UI layer.

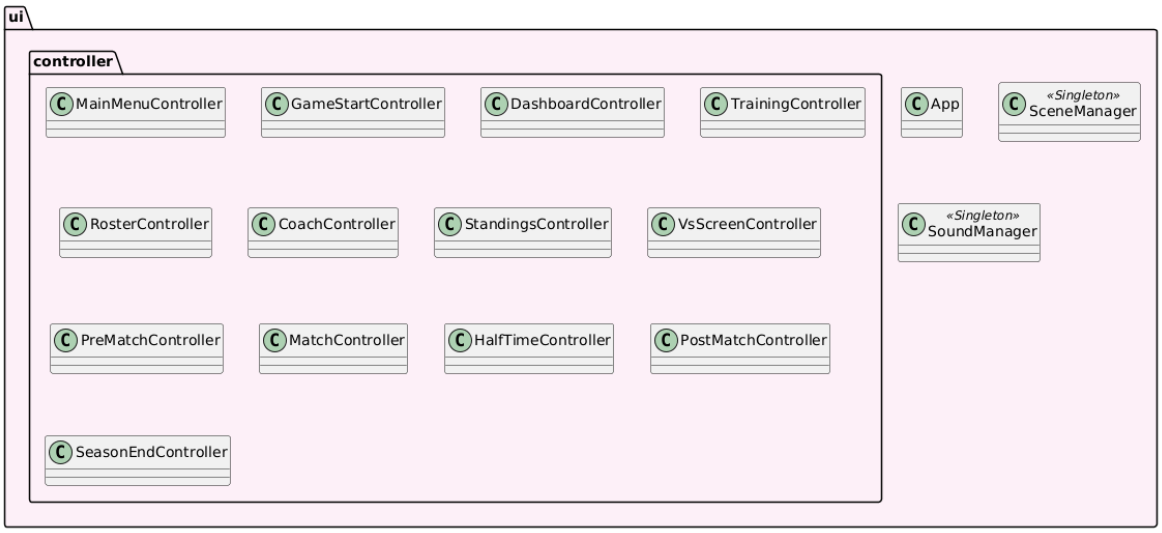

Figure 3.2: UI-layer architecture of the Sports Management Simulation Game

3.1.2 Handball Plugin – Proof of the Plugin Architecture

Package: `handball`

The addition of Handball served as a practical validation of the plugin architecture. A new handball package was created with seven classes implementing the sport plugin interfaces defined by the Sport API. No existing business logic or UI behaviour needed to be redesigned; only a single registration entry was added in DefaultGameFacade. The full class breakdown is provided in Section 6.4.

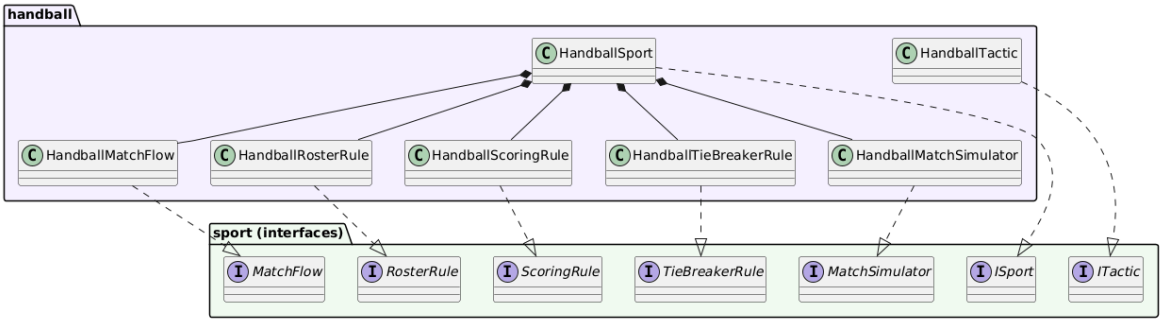

Figure 3.3: Handball plugin class structure

3.2 Design Principles

3.2.1 SOLID Principles

- **Single Responsibility Principle (SRP):** Each class has a clearly defined responsibility. For example, TrainingManager applies training effects, MatchManager handles match simulation, and PlayerGenerator creates player objects. No class is responsible for multiple unrelated tasks.
- **Open/Closed Principle (OCP):** The system is designed to support extension without requiring major modifications to existing code. Adding a new sport requires a new package implementing the sport plugin interfaces together with a registration entry in the SPORT_REGISTRY map inside DefaultGameFacade. The UI selection menu, game initialisation, and save/load restoration all derive their sport list from this registry.
- **Liskov Substitution Principle (LSP):** Any ISport implementation can replace another without affecting the calling code. MatchManager and DefaultGameFacade store references using the ISport interface, so switching from Football to Handball automatically changes league size, roster rules, and match simulation behaviour without additional conditional logic.

- **Interface Segregation Principle (ISP):** The plugin layer is divided into several smaller interfaces (MatchFlow, MatchSimulator, RosterRule, ScoringRule, and TieBreakerRule) rather than one large interface. Each class depends only on the functionality it actually uses.
- **Dependency Inversion Principle (DIP):** DefaultGameFacade depends on the ISport abstraction rather than concrete sport classes. Similarly, the UI layer depends on the GameFacade interface instead of directly depending on DefaultGameFacade.

3.2.2 Facade Pattern

The GameFacade interface provides a unified access point to the core game systems. The UI layer calls methods such as startNewGame(), applyTraining(), playPeriod(), and submitWeekResults() without interacting directly with managers, generators, or domain objects. This hides the complexity of the application layer and keeps the UI code focused and maintainable.

3.2.3 Singleton Pattern

The SceneManager and SoundManager classes use the Singleton pattern. A single instance of each class exists throughout the lifetime of the application, ensuring that all controllers share the same stage and audio engine.

Chapter 4

User Interface Design

4.1 UI Overview

The user interface is built entirely in Java using JavaFX. All layouts are created programmatically inside controller classes instead of using FXML. A single CSS file (`style.css`) defines the shared dark theme used across all screens.

4.2 Screen Descriptions

For a detailed walkthrough of all 13 screens refer to the accompanying User Manual. Table 4.1 lists each screen together with its controller class and primary purpose.

Table 4.1: Screens and their corresponding controller classes

Screen	Controller Class	Purpose
Main Menu	`MainMenuController`	Start a new game or load from one of three save slots
New Game	`GameStartController`	Enter manager name, team name, gender, and sport
Dashboard	`DashboardController`	Weekly hub: upcoming match, standings, coach and injury cards
Training	`TrainingController`	Choose training intensity (Light / Medium / Hard)
VS	`VsController`	Full-screen match announcement before each game
Pre-Match	`PreMatchController`	Squad and tactic review before kick-off
Match	`MatchController`	Period-by-period play, live score, player status bars
Half-Time	`HalfTimeController`	Tactical changes between the two halves
Post-Match	`PostMatchController`	Final result, injuries, energy changes, reputation gained
Standings	`StandingsController`	Full league table with user team highlighted
Squad Management	`RosterController`	Move players between starting lineup and substitutes
Coach	`CoachController`	Hire coaches unlocked by season number and reputation
Season End	`SeasonEndController`	Champion display, final standings, start next season

4.3 UI Architecture

4.3.1 SceneManager

The SceneManager is a Singleton responsible for managing the JavaFX Stage. All screen transitions pass through its switchTo(String screenName, GameFacade facade) method. The method uses a switch statement to create the appropriate controller and calls getRoot() to retrieve the corresponding Parent node. Instead of creating a new Stage for each screen, SceneManager replaces the root of the existing Scene. This keeps the application window active and preserves the attached CSS stylesheet.

```
public void switchTo(String screenName, GameFacade facade) {
    Parent root;
    switch (screenName) {
        case "dashboard":
            root = new DashboardController(facade).getRoot();
            break;
        case "match":
            root = new MatchController(facade).getRoot();
            break;
        // ... other cases
        default:
            throw new IllegalArgumentException("Unknown screen:
   "
            + screenName);
    }
    stage.getScene().setRoot(root);
    stage.show();
}
```

Listing 4.1: SceneManager switchTo method (excerpt)

4.3.2 Controllers

The project uses a simple controller-based JavaFX architecture instead of FXML-based MVC. Each controller class in the ui.controller package receives a GameFacade reference when it is created. Its main public method is getRoot(), which builds the layout programmatically and returns it. This keeps controllers separate and easy to replace or modify.

4.3.3 Visual Design and User Experience

The interface uses a consistent dark theme with green, blue, orange, and red accent colours. Green is used for positive states such as wins and highlighted standings, orange is used for warnings such as low energy, and red is used for injured players. This colour system helps the player quickly understand squad status without reading every value individually.

Injured players are marked with a badge on the Match and Squad screens, and the Dashboard shows a dedicated injury card whenever a player is injured. Sound effects are also used to improve feedback. Button clicks, victories, draws, game-over events, and championship wins each have their own audio cue. Together, these design choices make the interface feel more responsive without relying on animations.

4.3.4 UI Development Challenges

Building the interface revealed several problems that were not obvious during the initial architecture design.

Scene transitions and state management. Each screen is rebuilt whenever the player navigates to it. Any data not stored in the facade is lost during the transition. This caused the Dashboard to request current match data before the week had started, which crashed the screen transition. The solution was to add a separate read-only facade method so the Dashboard could display upcoming match information without changing the game state.

Layout stability with dynamic content. ComboBox controls on the New Game screen caused nearby panels to resize when the dropdown menu opened. This happened because HBox distributes extra space based on preferred component widths. Replacing the layout with a GridPane using percentage-based column widths fixed the problem and prevented layout shifting.

Inter-screen data passing. Information shared between screens (such as the match result shown on the Post-Match screen) cannot be passed directly between controllers. Instead, this data must be stored in the facade or application state. This design keeps controllers independent and prevents direct dependencies between screens.

Chapter 5

Application Flow

The overall game follows a clear flow from start to finish.

1. **Game Setup**: The player enters a manager name, team name, gender, and sport type. DefaultGameFacade.startNewGame() creates the league, AI teams, and fixture schedule, then starts Season 1.
2. **Weekly Cycle (repeated each week)**: Each week the player goes through training, squad management, and the match. After the match, results are processed, standings are updated, and the week advances.
3. **Season End**: When all match weeks are completed, the game recognises the season as finished and displays the Season End screen with the champion. The player can then start a new season using the same teams and updated player states.
4. **Save / Load**: From the Dashboard, the player can save the game to one of three save slots. The full game state is serialised into a JSON file using Gson. Loading restores standings, week number, player attributes, coach data, and tactics.

Chapter 6

Implementation Details

6.1 GameFacade

GameFacade is a Java interface that defines the operations available to the UI layer. It is implemented by DefaultGameFacade. The interface is organised into several logical groups:

- Sport discovery (`getAvailableSportNames`) returns the list of available sports so the UI can populate its selection menu without referencing concrete sport classes
- Game initialisation (`startNewGame`)
- Profile and team queries
- Season management (`startNewSeason`, `isSeasonComplete`)
- Weekly cycle operations (`startWeek`, `applyTraining`, `playPeriod`, `submitWeekResults`)
- Standings and week queries
- Team management (`addPlayerToStarting`, `setTactic`, `setCoach`)
- Save and load operations

This design allows `DefaultGameFacade` to be replaced with another implementation (such as a networked version or a test stub) without requiring changes to the UI code.

6.2 Managers

6.2.1 MatchManager

`MatchManager` handles all match-related operations. Its main responsibilities are:

- **simulateAIMatches**: Loops through all matches in a MatchWeek, skips the user's match, and simulates the remaining AI matches.

- **playUserPeriod**: Simulates a single match period using the sport's `MatchSimulator` implementation.
- **applyPostMatchEffects**: Applies post-match effects such as condition changes, energy loss, injury risk increases, random injuries for high-risk players, and injury recovery progression.

6.2.2 TrainingManager

`TrainingManager` manages player training and weekly recovery. Training effects are shown in Table 6.1. Coach bonuses increase condition gains and reduce energy loss based on the current coach-team relationship value (0–100).

Table 6.1: Training intensity effects (before coach multiplier)

Intensity	Energy Loss	Condition Gain	Injury Risk
Light	-5	+3	-2
Medium	-10	+6	0
Hard	-20	+10	+5

Before each new week, players recover energy based on their status: healthy starters recover 20 points, healthy substitutes recover 25 points, and injured players recover 10 points. Weekly recovery also lowers injury risk.

6.2.3 LeagueManager

LeagueManager creates a League object containing the user's team together with the required number of AI teams. It uses TeamGenerator to create AI teams with random names and generated player rosters. It also supports rebuilding a league from saved AI team names during the load process.

6.2.4 SeasonCycleManager

SeasonCycleManager tracks the current season, current week, and all previously completed seasons. It manages season creation, fixture access, and standings queries through the Season and LeagueTable domain objects. It also includes forceCompleteAndAdvance(), which is used during loading when a completed season is restored from a save file.

6.2.5 PlayerManager

PlayerManager is responsible for creating and storing player objects. It assigns a unique auto-incrementing ID to each player. Other classes create players through playerManager.createPlayer() instead of directly calling the Player constructor.

6.2.6 TeamManager

TeamManager handles changes made to the user's team. It validates roster updates (such as preventing duplicate players), updates tactics and coaches, and communicates with the Team domain object.

6.3 Generators

6.3.1 PlayerGenerator

PlayerGenerator creates Player objects with random attributes. Names are selected from two static pools in NamePool: one for male names and one for female names. Player age is randomly generated between 18 and 50. Energy, condition, and injury risk values are also generated within predefined ranges.

6.3.2 TeamGenerator

TeamGenerator creates Team objects for AI-controlled opponents. It generates a team with a random name and fills the roster with generated players of the correct gender using PlayerGenerator.

6.4 Sport Plugins

6.4.1 Football Plugin

The FootballSport class combines all football-specific components:

- **FootballMatchFlow**: Defines 2 periods per match.
- **FootballRosterRule**: Defines the valid roster size (11 starters).
- **FootballScoringRule**: Calculates goals using team strength and random factors.
- **FootballTieBreakerRule**: Uses goal difference and goals scored to break ties.
- **FootballMatchSimulator**: Controls football match simulation using the components above.

- **FootballTactic**: Provides OFFENSIVE, BALANCED, and DEFENSIVE tactic options.

6.4.2 Handball Plugin

The HandballSport class follows the same structure with handball-specific rules:

- **HandballMatchFlow**: Defines 2 periods per match.
- **HandballRosterRule**: Defines the valid roster size (7 starters).
- **HandballScoringRule**: Produces higher-scoring results typical of handball.
- **HandballTieBreakerRule**: Handles tie-breaking for the handball league.
- **HandballMatchSimulator**: Controls handball match simulation.

The architecture was designed to support additional sports. Adding a third sport requires only a new package and a registration entry, without major changes to existing code.

6.5 Save and Load System

6.5.1 GameState

`GameState` is a plain Java object that stores all information required to restore a saved game. It contains:

- Manager name, reputation, and season number.
- Sport name, gender, team name, and coach relationship.
- Coach details (name, level, required season, required reputation).
- Tactic style (stored as the string name of the `PlayStyle` enum).
- Player data for starters and substitutes (name, age, energy, condition, injury risk, injury status, and remaining injury duration).
- AI team names.
- Current week number and total number of weeks.
- Full standings data for every team.
- Season completion flag.
- Save date and time.

6.5.2 SaveLoadManager

SaveLoadManager uses the Gson library to serialise GameState objects into JSON files named save_slot_{id}.json. During loading, the file is read, deserialised, and converted back into a GameState object. The manager also checks all three save slots and generates slot summaries (save date or "Empty") for display in the UI.

The DefaultGameFacade.restoreFromGameState() method rebuilds managers, teams, and standings from the loaded state, restoring the game to its previous state.

Chapter 7

Technical Details

7.1 Tools and Technologies

Table 7.1: Technologies used in the project

Technology	Version	Purpose
Java	26	Core programming language
JavaFX	26	Desktop user interface framework
Maven	3.x	Build tool and dependency management
JUnit 5	5.10.2	Unit testing framework
Gson	2.13.2	JSON serialisation for save/load
maven-shade-plugin	3.5.1	Fat JAR packaging for distribution
IntelliJ IDEA	Latest	Development environment

7.2 Project Structure

```
src/
  main/
    java/
      application/    - GameFacade, managers, generators, save/load
      domain/         - Core business objects (Player, Team, Season, ...)
      football/       - Football plugin implementation
      handball/       - Handball plugin implementation
      sport/          - Plugin interfaces (ISport, MatchSimulator, ...)
      ui/             - App, SceneManager, SoundManager
      ui/controller/  - One controller per screen
    resources/
      style.css       - Global CSS theme
      images/         - Image assets (vs.jpg)
      sounds/         - Audio files (.m4a)
  test/
```

```
java/
  application/    - MatchManagerTest, TrainingManagerTest
  domain/         - PlayerTest, TeamTest, CoachTest
  football/       - FootballScoringRuleTest
  handball/       - HandballScoringRuleTest, HandballMatchSimulatorTest, ...
```

7.3 Module System

The project uses the Java module system through module-info.java. The module configuration includes dependencies for javafx.controls, javafx.fxml, javafx.media, and com.google.gson.

Five packages are opened to Gson during save/load operations: application, domain, football, handball, and sport. This allows Gson to access player and team data stored inside the GameState object during JSON serialisation and deserialisation.

7.4 Build and Run with Maven

All build and run operations are performed using Apache Maven from the project root directory.

```
//Compile all source files
mvn compile

//Run all JUnit 5 tests
mvn test

//Launch the JavaFX application
mvn javafx:run
```

7.5 Standalone Distribution

The project uses the maven-shade-plugin to create a fat JAR containing all required dependencies, including JavaFX and Gson. A Launcher class (application.Launcher) is used as the entry point to avoid JavaFX module-path issues when running the application outside a modular environment.

```
//Package into a fat JAR (skipping tests)
mvn clean package -DskipTests
```

This command generates:

`target/sports-manager-1.0-SNAPSHOT.jar`

A run.bat file is included in the project root to launch the application on Windows:

```
java --enable-native-access=ALL-UNNAMED -jar sports-manager-1.0-
    SNAPSHOT.jar
```

7.6 Unit Tests

The test suite covers the most important business logic components. Tests are written with JUnit 5 and follow the AAA pattern (Arrange, Act, Assert).

Systematic unit testing was particularly valuable in this project because much of the game logic involves numeric calculations with subtle boundary conditions. This includes energy loss, condition clamping, injury probability, and training multipliers. These behaviours are difficult to verify through manual gameplay alone. Unit tests made these rules explicit and helped detect several incorrect values before the UI was available.

The test suite also played an important role in verifying that each sport plugin followed the same behavioural rules. The Handball test classes confirmed that roster validation, scoring ranges, and match simulation behaved correctly before any Handball screen was implemented.

Table 7.2: Test classes and what they test

Test Class	What is tested
`PlayerTest`	Player attribute clamping, injury logic, recovery
`TeamTest`	Roster management, squad validation
`CoachTest`	Coach unlock conditions, multiplier calculations
`LeagueManagerTest`	League creation, team count, user team assignment
`MatchManagerTest`	Energy loss, injury risk, condition change
`TrainingManagerTest`	Training effects, weekly recovery
`FixtureTest`	Fixture generation, match week structure
`LeagueTableTest`	Standings update, sorting, tie-breaking
`FootballScoringRuleTest`	Score generation with different team strengths
`FootballTieBreakerRuleTest`	Football standings tie-breaking logic
`HandballScoringRuleTest`	Handball scoring characteristics
`HandballRosterRuleTest`	Handball roster validation (7 starters)
`HandballMatchSimulatorTest`	Full handball match simulation

Chapter 8

Changes from the M1 Design

The M1 document established the four-layer architecture, the sport plugin concept, the GameFacade as the single UI boundary, and the core domain entities. These decisions remained stable throughout the project. However, several parts of the system changed or expanded during implementation.

8.1 ITactic Interface and a New ISport Method Were Added to the Sport API

The M1 design already defined six Sport API interfaces (ISport, MatchSimulator, MatchFlow, RosterRule, ScoringRule, and TieBreakerRule), which remained stable throughout implementation. Two additions were made during development that were not part of the original design.

First, a seventh interface named ITactic was introduced to represent the play style used during a match. This was necessary because Football and Handball use different tactic options, while the application layer cannot directly depend on sport-specific tactic classes.

Second, a getDefaultTactic() method was added to ISport so that LeagueManager could assign the correct default tactic when creating new teams, without depending on concrete tactic implementations.

The M1 text also described ScoringRule as determining how many league points are awarded for a win, draw, or loss. During implementation this responsibility was adjusted. ScoringRule generates the goals scored during a match, while league points (3 for a win, 1 for a draw, 0 for a loss) are calculated by LeagueTable and remain the same for every sport.

8.2 Two Planned Plugin Components Were Consolidated into Core Domain Logic

The M1 Sport API diagram showed InjurySystem and SubstitutionManager as separate classes inside the Football plugin. In the final implementation, injury logic was moved into MatchManager and the Player class, while substitutions are handled directly by the Team class. Extracting these into separate plugin components would have increased interface complexity without providing significant architectural benefits at this project scale.

8.3 Single MainWindow Became 13 Controllers

The M1 architecture diagram showed a single MainWindow class representing the entire UI layer. The final implementation contains 13 dedicated controller classes together with a SceneManager singleton, a SoundManager singleton, and an App entry point. The final UI implementation was much larger than originally anticipated in M1.

8.4 New Application-Layer Classes Were Added

The M1 design included LeagueManager, TeamManager, and MatchManager, but it did not include PlayerManager, PlayerGenerator, TeamGenerator, TrainingManager, or SeasonCycleManager. These classes were added to keep responsibilities properly separated.

PlayerManager was particularly important for the save/load system because every player required a stable unique identifier across save and load operations.

8.5 League Configuration Made Dynamic

The M1 domain model showed that a Season could contain multiple leagues. In the final implementation, each season consistently contains a single league.

Additionally, league size and fixture length were originally hardcoded for football. This was later refactored so that the team count and schedule length are determined by the sport itself. This change allowed the 12-team, 22-week handball league to work correctly without requiring separate logic.

8.6 Three Major Systems Were Added During M3

The save/load system, coach progression system, and manager reputation system were not part of the original M1 design. All three were introduced during M3. Their implementation details and the problems encountered during development are discussed in the Improvements from M2 and Post-Mortem chapters.

8.7 Sport Selection Was Refactored

The original startNewGame() method was designed to accept an ISport object directly, which required the UI layer to instantiate concrete sport classes. This conflicted with the layered architecture principle defined in M1.

During M3, the method signature was changed to accept a sport name string instead. A central sport registry was also introduced inside DefaultGameFacade so that sport creation became fully centralised.

As a result, the UI sport selection menu and the save/load restore process both derive their sport list from the registry instead of hardcoded references.

Chapter 9

Improvements from M2

M3 completed the system by adding several features that were missing or incomplete in M2:

- **Complete JavaFX UI**: A full set of screens covering every stage of the game, including navigation, interaction, and dark-themed CSS styling.
- **Handball Plugin**: A complete handball sport package implementing all seven sport interfaces. M2 included only the Football plugin.
- **Save/Load System**: A GameState snapshot class and a SaveLoadManager serialise the full game state into JSON files using Gson. Loading restores standings, player attributes, coach data, week number, and tactics. Three save slots are available from the Dashboard.
- **Coach System**: Five coaches with level-based unlock requirements linked to manager reputation and season number. Each coach provides a training bonus and reduces match energy loss. Bonus effectiveness is scaled using a coach-team relationship value (0–100) that increases with wins and decreases with losses.
- **Manager Reputation**: Reputation points are awarded after each match (win: +4, draw: +1) and carry across seasons. Reputation is used to unlock higher-level coaches.
- **Sound System**: Audio feedback is used for button clicks, match victories, draws, game-over events, and season champion celebrations.
- **Injury Report on Dashboard**: A dedicated injury card appears on the Dashboard whenever a squad member is injured, helping the player notice injuries before selecting the lineup.
- **Post-Match Condition System**: Player condition changes after every match based on the result. Wins provide a small bonus, while losses apply a larger penalty in addition to the weekly training effect.

Chapter 10

Post-Mortem

10.1 What Went Right

- **The plugin architecture worked successfully.** Adding the Handball sport plugin required no major changes to the domain, application, or UI layers. Only a new package containing seven classes was added. This confirmed the main architectural goal established in M1.
- **The GameFacade boundary supported parallel development.** Different team members were able to work on UI screens and backend systems at the same time with minimal conflicts. The facade interface provided a clear contract that allowed UI development to continue independently from backend implementation details.
- **Domain entities remained stable from M1.** Core classes such as Player, Team, League, LeagueTable, Fixture, and Match required very few structural changes compared to the original design. The early modelling effort proved useful throughout all three milestones.
- **Unit tests identified important defects before the UI was available.** The JUnit 5 test suite detected a critical match lifecycle ordering issue, incorrect condition clamping, and a duplicate state update that would have been difficult to isolate through manual UI testing.
- **Dynamic league configuration adapted cleanly to multiple sports.** When Handball was added, neither the league creation logic nor the fixture generation system required modification. Both the 18-team football league and the 12-team handball league worked correctly using the same code structure. This confirmed that allowing each sport implementation to define its own league configuration was a good architectural decision.

10.2 What Went Wrong

- **The save/load system required several bug fixes.** Because save/load was not planned during M1, the GameState class was initially created without a complete understanding of all required data. This caused several problems: loading always reset the game to Week 1, standings disappeared because AI team names were not saved, and injury states were not restored correctly during seasonal loading. Fixing these issues required adding new fields and methods to classes that had already been completed.
- **Duplicate state updates caused gameplay problems.** Weekly energy recovery was triggered in two different places: inside DefaultGameFacade and again on the post-match screen. This caused players to recover energy twice per week, making the game too easy. A similar issue caused substitute condition changes to be applied twice. These bugs happened because responsibilities were not fully separated, allowing the same logic to run in multiple places.
- **The Dashboard crashed when opened before a week had started.** The current-match accessor was called before startWeek() had executed, producing a null reference that crashed the scene transition during the first navigation to the Dashboard. The main issue was that the facade could not provide upcoming match data without also changing game state. A separate read-only query method was added to GameFacade to solve this problem.
- **Starting a new game in the same session crashed the application.** The league manager threw an exception when the player returned to the main menu and started another game without restarting the application. Session reuse had not been considered during M1, where the design assumed only one game session per application launch.
- **Handball rosters produced duplicate player names.** The original name pool was designed for a single 18-player football squad. Creating additional handball teams could produce repeated player names. The solution was to generate all player names from one shuffled pool instead of generating them separately for each team.
- **Game balance required several adjustment iterations.** Coach bonuses, reputation gain, injury risk, energy loss, and scoring ranges all required tuning after the full gameplay loop became playable. No balance targets had been defined during M1 or M2, which increased the number of late-stage balancing changes.

Chapter 11

Future Work

- **Transfer market**: Allow players to be signed and released between seasons, adding budget management to the game.
- **Player progression**: Add experience points and skill levels so players can improve or decline with age across multiple seasons.
- **Animated match view**: Display a visual match simulation showing events such as shots, fouls, and substitutions instead of showing only the final result.
- **AI difficulty levels**: Give AI teams different strength levels and tactical behaviour to create varied gameplay difficulty.
- **Additional sports**: Sports such as Basketball, Volleyball, and Ice Hockey could be added as future sport plugins because the architecture already supports extension.
- **Tournament mode**: Add a cup competition running alongside the regular league season.
- **Statistics screen**: Display career statistics for the manager and individual players across multiple seasons.
- **Responsive UI**: Make the application window resizable using VBox.setVgrow and HBox.setHgrow constraints so the layout adapts to different screen sizes.
- **Database persistence**: Replace JSON save files with a local SQLite database for more reliable and structured storage.
- **Localisation**: Add internationalisation support so the game can support multiple languages.

Chapter 12

Conclusion

This project demonstrates how a moderately complex application can be developed using object-oriented principles and a layered architecture. The four-layer structure (Domain, Application, Plugin, and UI) kept responsibilities separated and helped maintain a clear system design. The Facade pattern kept the UI layer independent from the business logic, while the plugin architecture made it possible to support multiple sports without major architectural changes.

M3 expanded the project from the working backend system developed in M2 into a fully playable game with a complete JavaFX interface, sound feedback, a coach and reputation progression system, save/load support, and a second sport. Most of the architectural decisions made during M1 remained stable throughout implementation. The main changes from the original design are discussed in Chapter 8.

The final system is a functional sports management simulation that demonstrates several software engineering concepts, including SOLID principles, Separation of Concerns, the Facade and Singleton design patterns, plugin-based extensibility, persistent state management, and systematic unit testing of core business logic.